MCMC & Hamiltonian Monte Carlo

Cutting-Edge Stochastic Approximation

Edward Milsom

2026-10-15

Slide 1 In this Episode ...

- Recap: Expectations and Monte Carlo approximation
- Markov-Chain Monte Carlo (MCMC)
- The Metropolis(-Hastings) method in detail
  - algorithm
  - justification and observations
  - examples and effect of dimensionality
- Hamiltonian Monte Carlo (HMC)
- Further reading and *Python* software libraries for HMC
- Not covered in this course, but well-known: Gibbs sampling.

Slide 2 Recap — Approximating Expectations

- Bayesian integrals are often analytically intractable, *e.g.* making predictions for new data y_* with model M:

$$p\left(y_*|\mathcal{D},\mathcal{M}\right) = \int p\left(y_*|\boldsymbol{\theta},\mathcal{M}\right) p\left(\boldsymbol{\theta}|\mathcal{D},\mathcal{M}\right) d\boldsymbol{\theta}$$

- This is an example of computing an expectation:

$$F = \langle f(\theta) \rangle_{p(\theta)} = \int f(\theta)\, p(\theta)\, d\theta$$

- These can be approximated by a Monte Carlo average:

$$\widehat{F} = \frac{1}{R}\sum_{r=1}^{R} f\left(\boldsymbol{\theta}^{(r)}\right)$$

- Where $\boldsymbol{\theta}^{(r)}$ is an independent random sample from $p(\boldsymbol{\theta})$

Slide 3 Recap — Monte Carlo Approximation

- In the last lecture, we considered methods for obtaining $\boldsymbol{\theta}^{(r)}$:
  - importance sampling
- In both cases we assumed $p(\boldsymbol{\theta}) = P^*(\boldsymbol{\theta})/Z$
  - we can compute $P^*(\boldsymbol{\theta})$ but not Z
- We then defined a proposal distribution:

$$Q(\boldsymbol{\theta}) = Q^*(\boldsymbol{\theta})/Z_Q$$

from which we can generate samples and evaluate $Q^*(\boldsymbol{\theta})$

- It is required that $Q(\boldsymbol{\theta})$ is close to $P(\boldsymbol{\theta})$

Markov Chain Monte Carlo

Slide 4 Markov Chain Monte Carlo (MCMC) — Motivation

- It is difficult to find a good proposal distribution $Q(\boldsymbol{\theta})$ that is close enough to $P(\boldsymbol{\theta})$ to be practical
- Especially true in high dimensions!
- Markov Chain Monte Carlo (MCMC) is a sampling method designed to obtain samples from $P(\boldsymbol{\theta})$ without need for a "globally close" $Q(\boldsymbol{\theta})$
- Instead, it makes use of a "local" proposal $Q\left(\boldsymbol{\theta}'|\boldsymbol{\theta}^{(t)}\right)$ that is conditional on the current sample (or state) $\boldsymbol{\theta}^{(t)}$
- To explain this, let's first define a Markov chain …

Slide 5 Markov Chains — Some Definitions

- A first-order Markov chain is a series of random variables $x^{(1)}, ..., x^{(T)}$ such that for all $t \in \{1, ..., T\}$:

$$p\left(x^{(t)}|x^{(1)}, ..., x^{(t-1)}\right) = p\left(x^{(t)}|x^{(t-1)}\right)$$

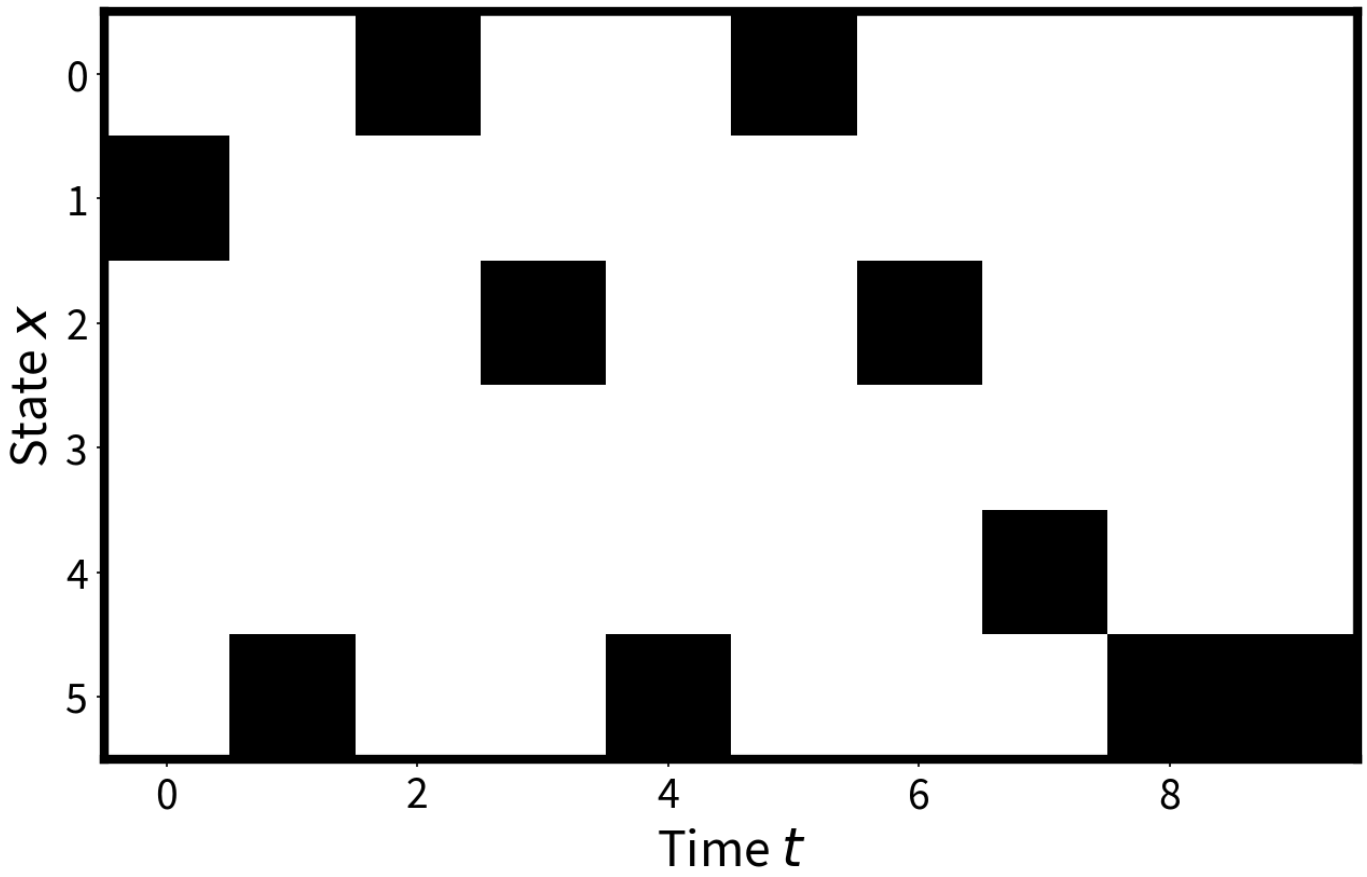
- This is known as a conditional independence property
- We can define a Markov chain by specifying:

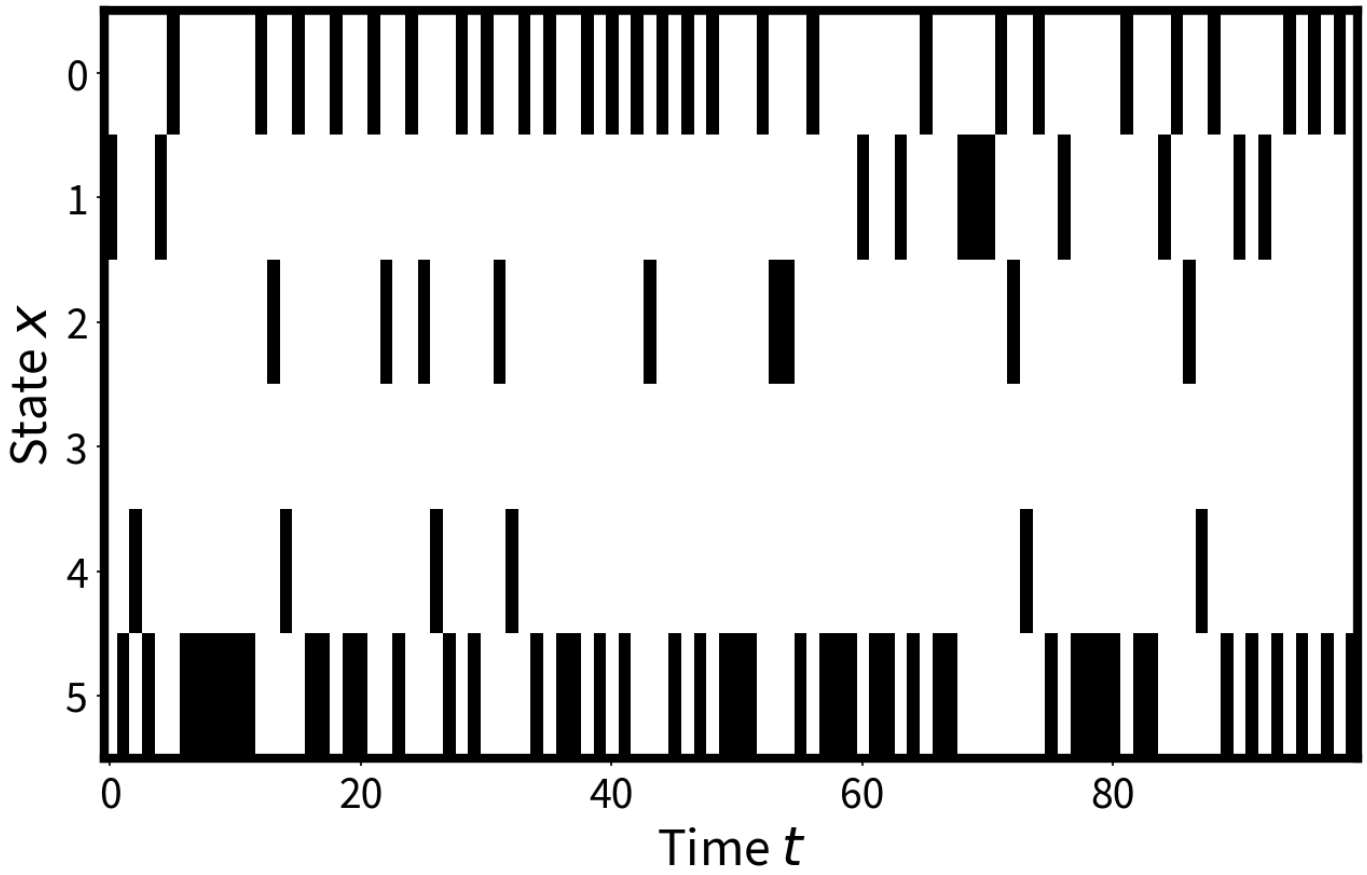
- a state-space, *e.g.* x or $\boldsymbol{\theta}$
- an initial state (distribution) $p\left(x^{(0)}\right)$
- the transition probabilities $T\left(x'|x\right)$ of going from x to x'

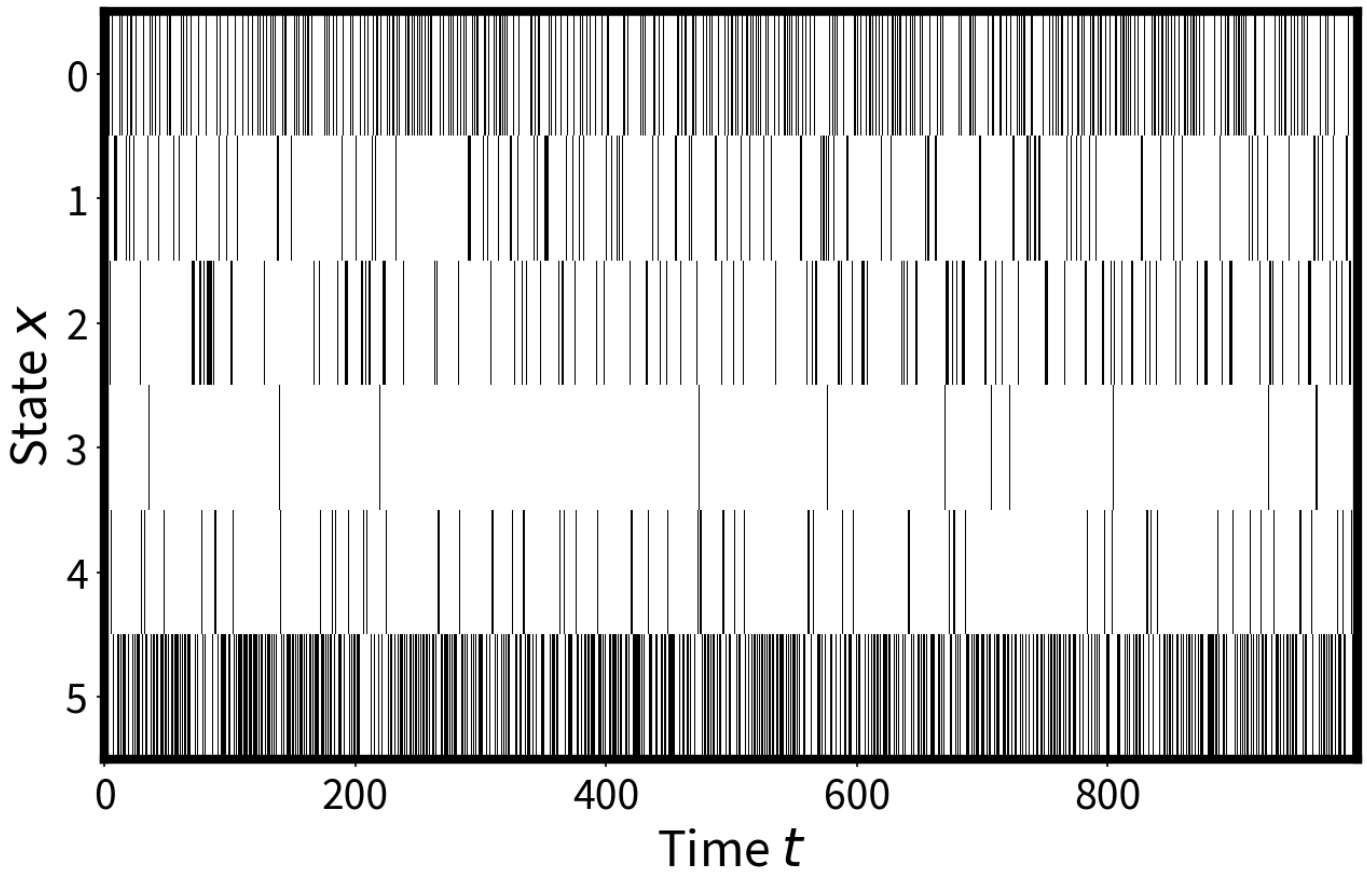
- Given certain conditions, as $t \to \infty$ the chain will converge on a unique equilibrium distribution
- We can exploit this to design a Markov chain whose equilibrium distribution is our desired quantity $p\left(\boldsymbol{\theta}\right)$, such as our posterior $p\left(\alpha, \sigma, r|\mathbf{y}\right)$

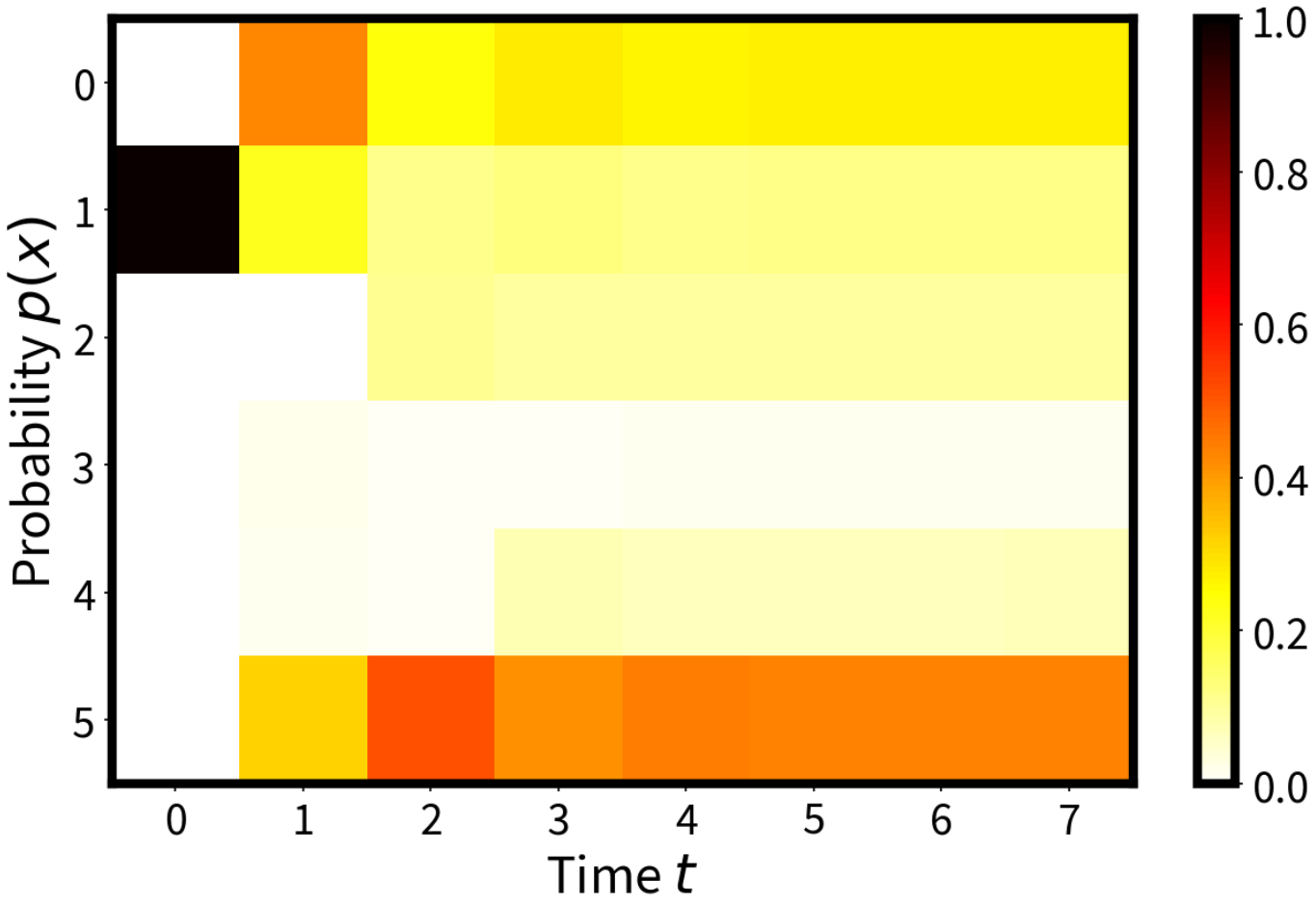
Slide 6 **Markov Chains — A Simple Illustration**

A simple Markov chain with 6 discrete states $x \in \{0, ..., 5\}$ and transition matrix $\mathbf{T}$:


State X
0
1
2
3
4
5
0
2
4
6
8
Time t


State X
0
1
2
3
4
5
0
20
40
60
80
Time t


State X
0
1
2
3
4
5
0
200
400
600
800
Time t



The Metropolis(-Hastings) Algorithm

Slide 7 The Metropolis-Hastings Algorithm — Basics

- Previously, we considered using a fixed, global, proposal density $Q(\theta)$ as a basis to generate Monte Carlo samples
- Instead, the Metropolis-Hastings method uses a probability density (or kernel) to propose a candidate state θ' that depends on the current state $\theta^{(t)}$:

$$Q\left(\theta' | \theta^{(t)}\right)$$

- It is *not* required that $Q\left(\theta' | \theta^{(t)}\right)$ be representative of $p(\theta)$

- *e.g.* $Q\left(\theta^{'}|\theta^{(t)}\right)$ could be a narrow Gaussian centred on $\theta^{(t)}$
- The new state $\theta^{'}$ may either be accepted or rejected
- The equilibrium distribution of the sequence of states thus generated should converge on our distribution of interest $p\left(\theta\right)$

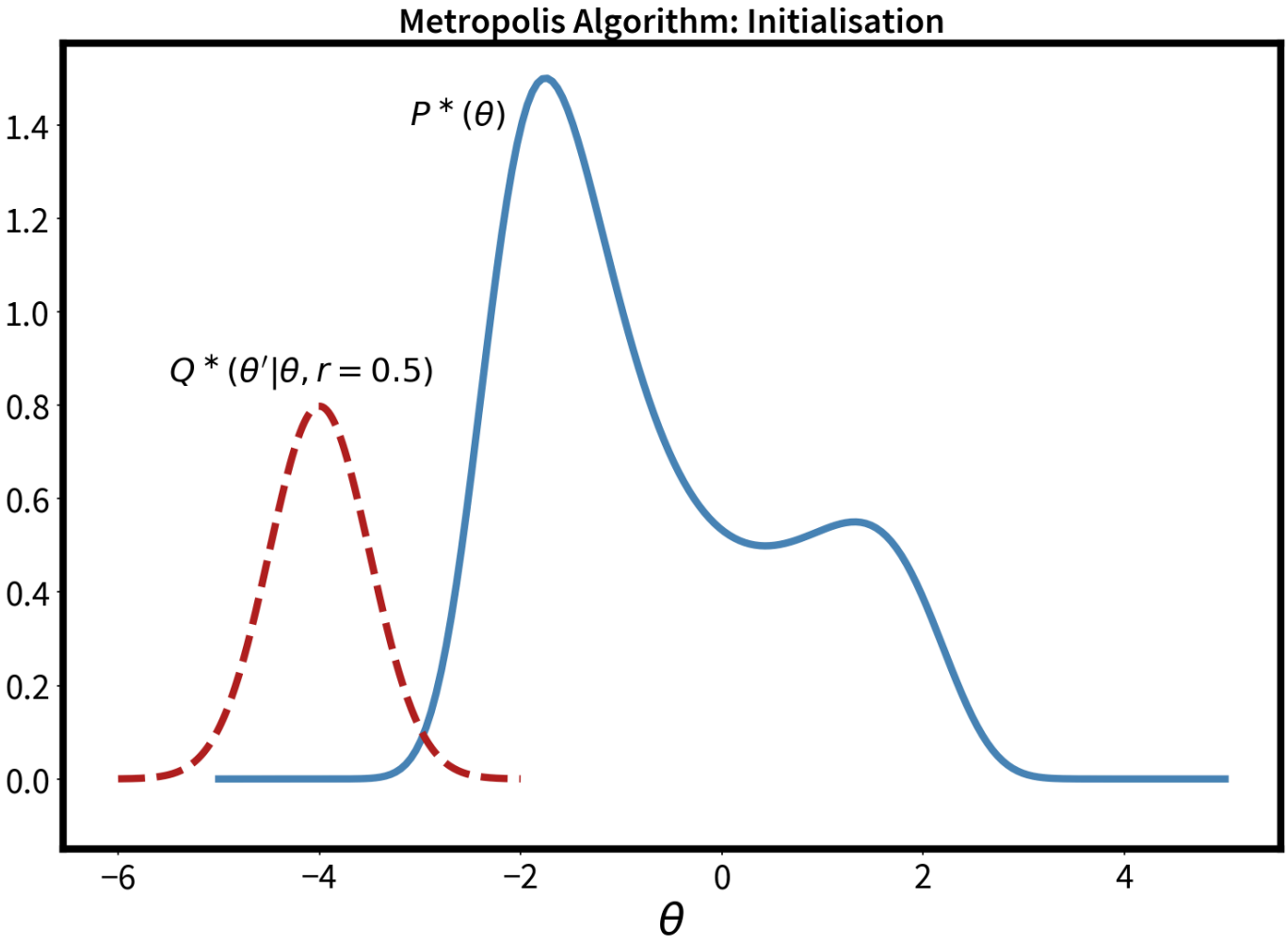
Slide 8 The Metropolis-Hastings Algorithm

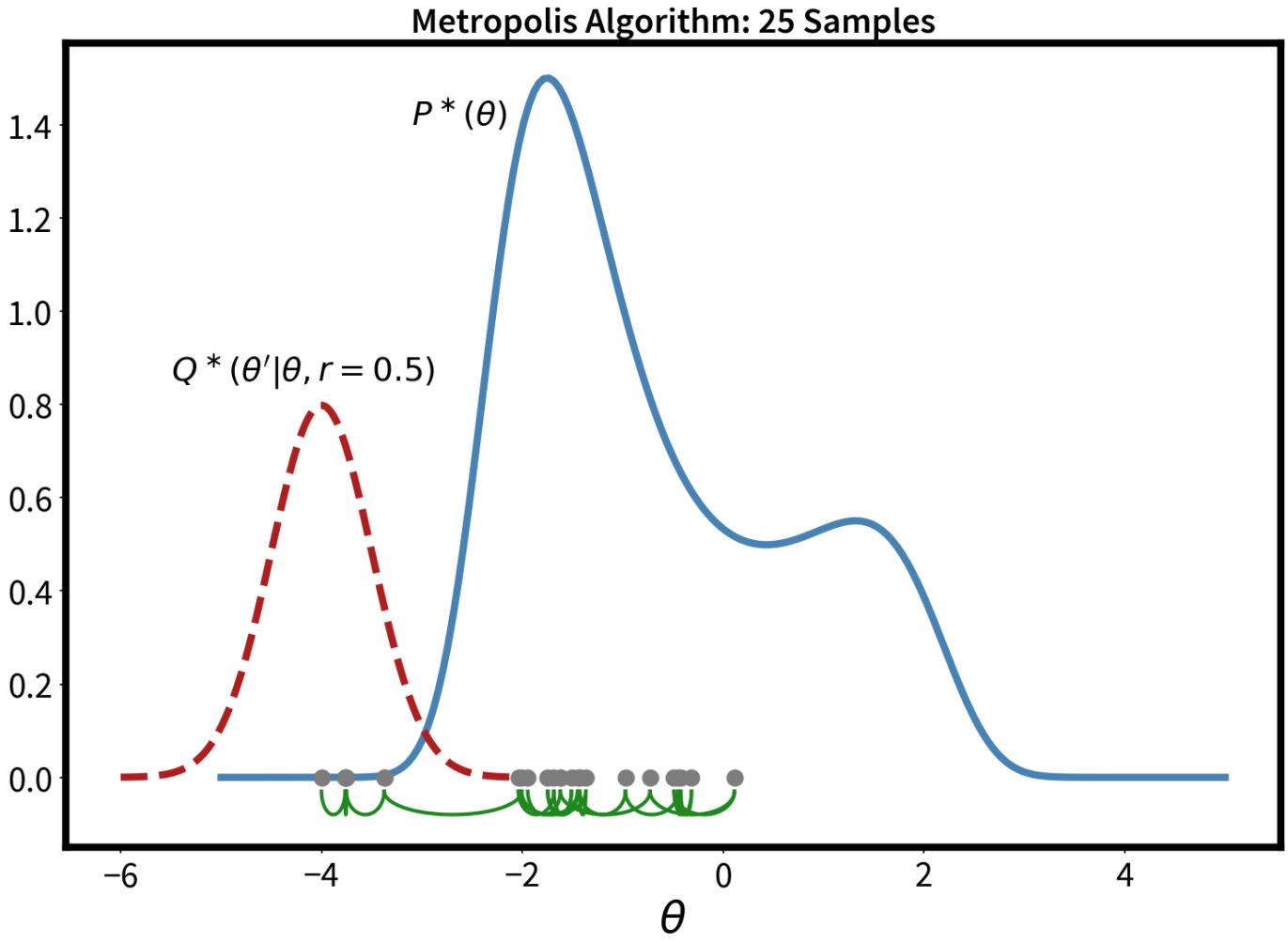
1. Select a starting point $\theta^{(t)}$ at $t = 0$
2. Generate proposal $\theta^{'}$ from $Q\left(\theta^{'}|\theta^{(t)}\right)$
3. Evaluate $P^{*}\left(\theta^{'}\right)$
4. Compute the threshold:

$$\kappa = \frac{P^{*}\left(\theta^{'}\right)}{P^{*}\left(\theta^{(t)}\right)}\frac{Q\left(\theta^{(t)}|\theta^{'}\right)}{Q\left(\theta^{'}|\theta^{(t)}\right)}$$

5. If $\kappa \geq 1$ then accept $\theta^{'}$
6. If $\kappa < 1$ then accept $\theta^{'}$ with probability κ
7. Set sample $\theta^{(t+1)}$ to $\theta^{'}$ if accepted, repeat previous $\theta^{(t)}$ if rejected
8. Return to Step

Slide 9 The Metropolis Algorithm — Example





Slide 10 The Metropolis Algorithm — Justification

- It may be demonstrated that, given some reasonable requirements, samples from the Metropolis algorithm converge on $p(\mathbf{\theta})$
- *e.g.* See MacKay, Chapter 29.6
- A key property is known as detailed balance:

$$T(\mathbf{\theta}_a|\mathbf{\theta}_b)\, p(\mathbf{\theta}_b) = T(\mathbf{\theta}_b|\mathbf{\theta}_a)\, p(\mathbf{\theta}_a) \quad \text{for all } \mathbf{\theta}_a, \mathbf{\theta}_b$$

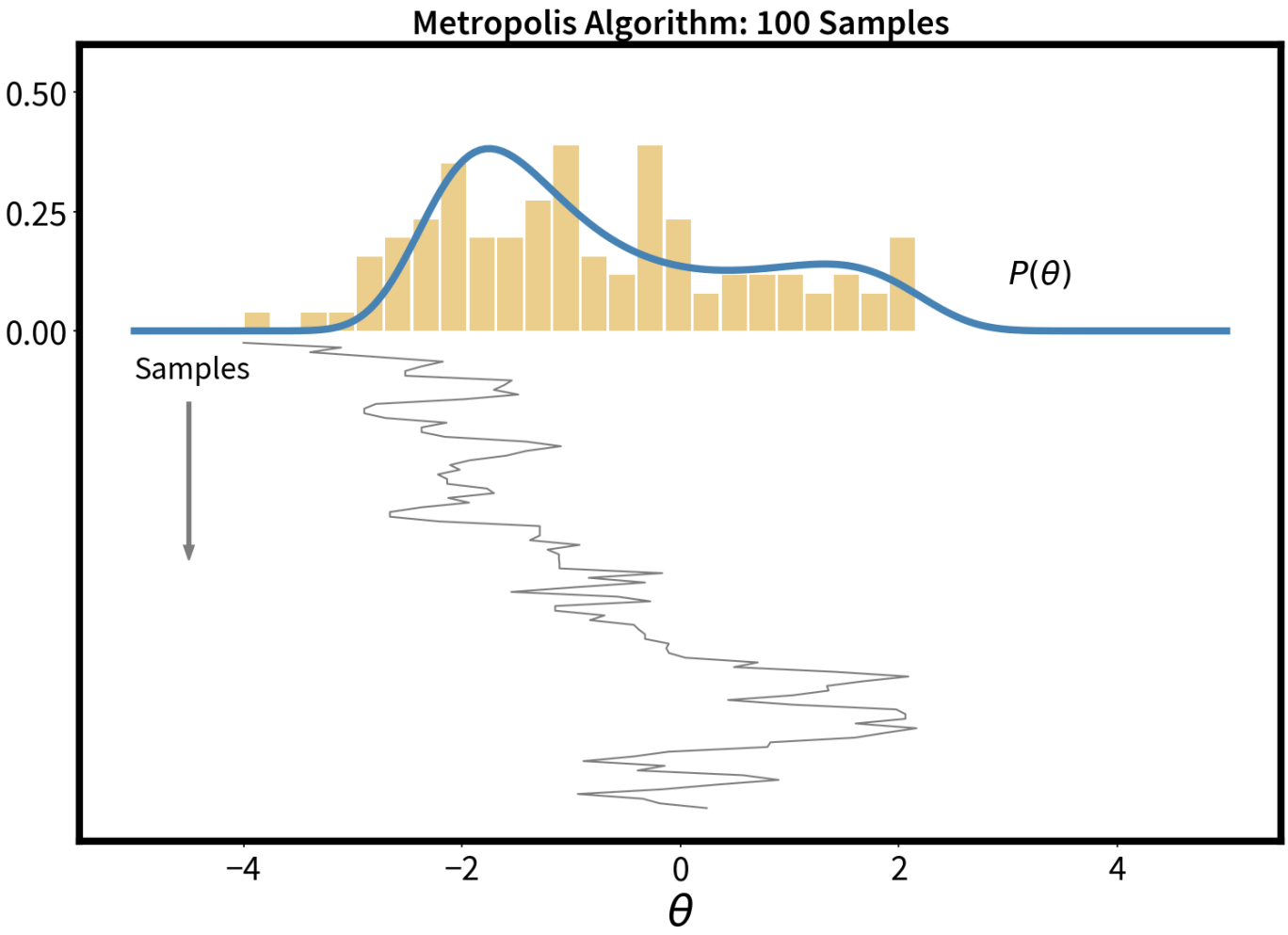
- This is a necessary condition for the chain to converge on the desired $p(\mathbf{\theta})$ and is satisfied by the Metropolis algorithm

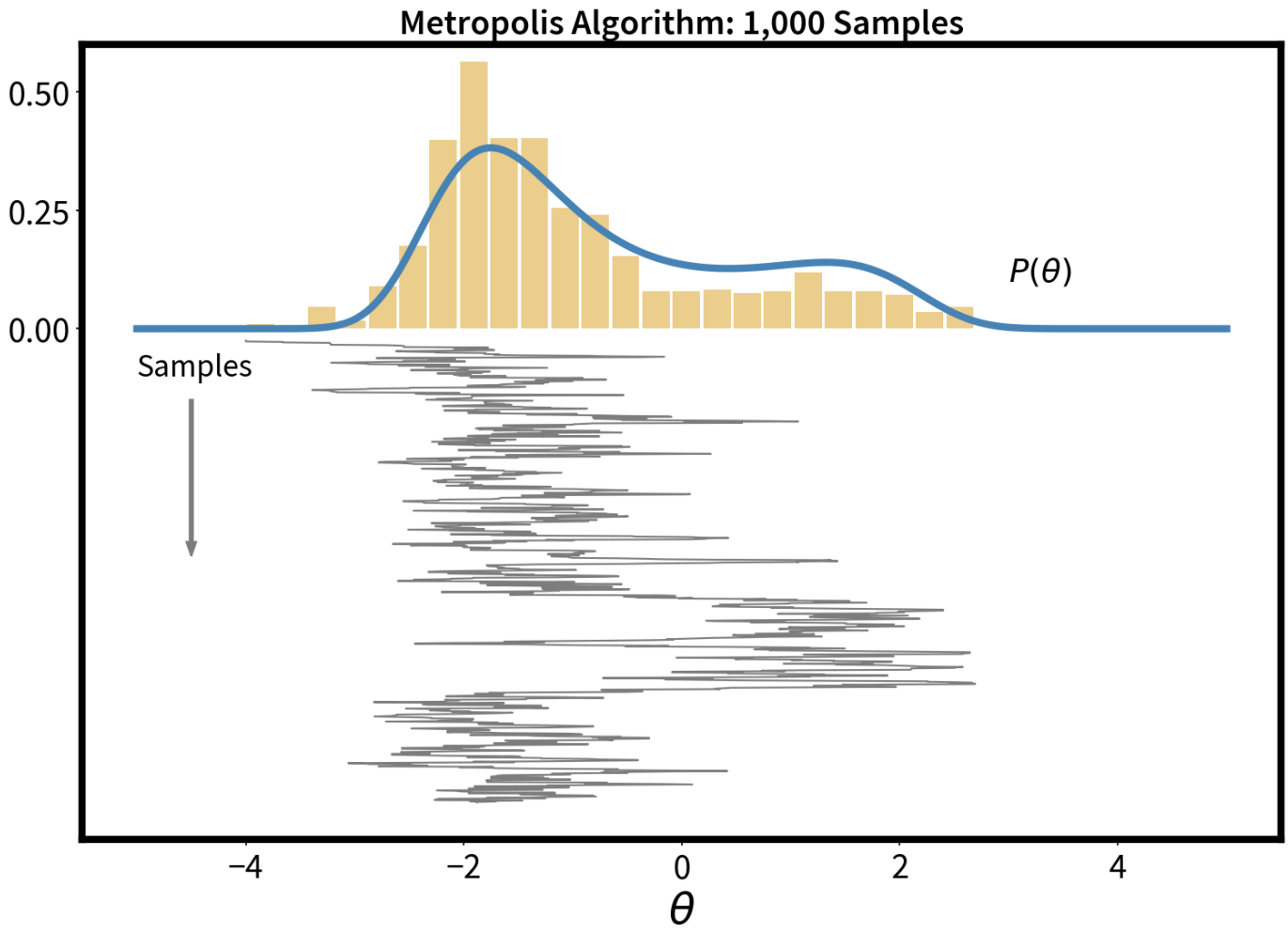
Slide 11 The Metropolis Algorithm — Comments

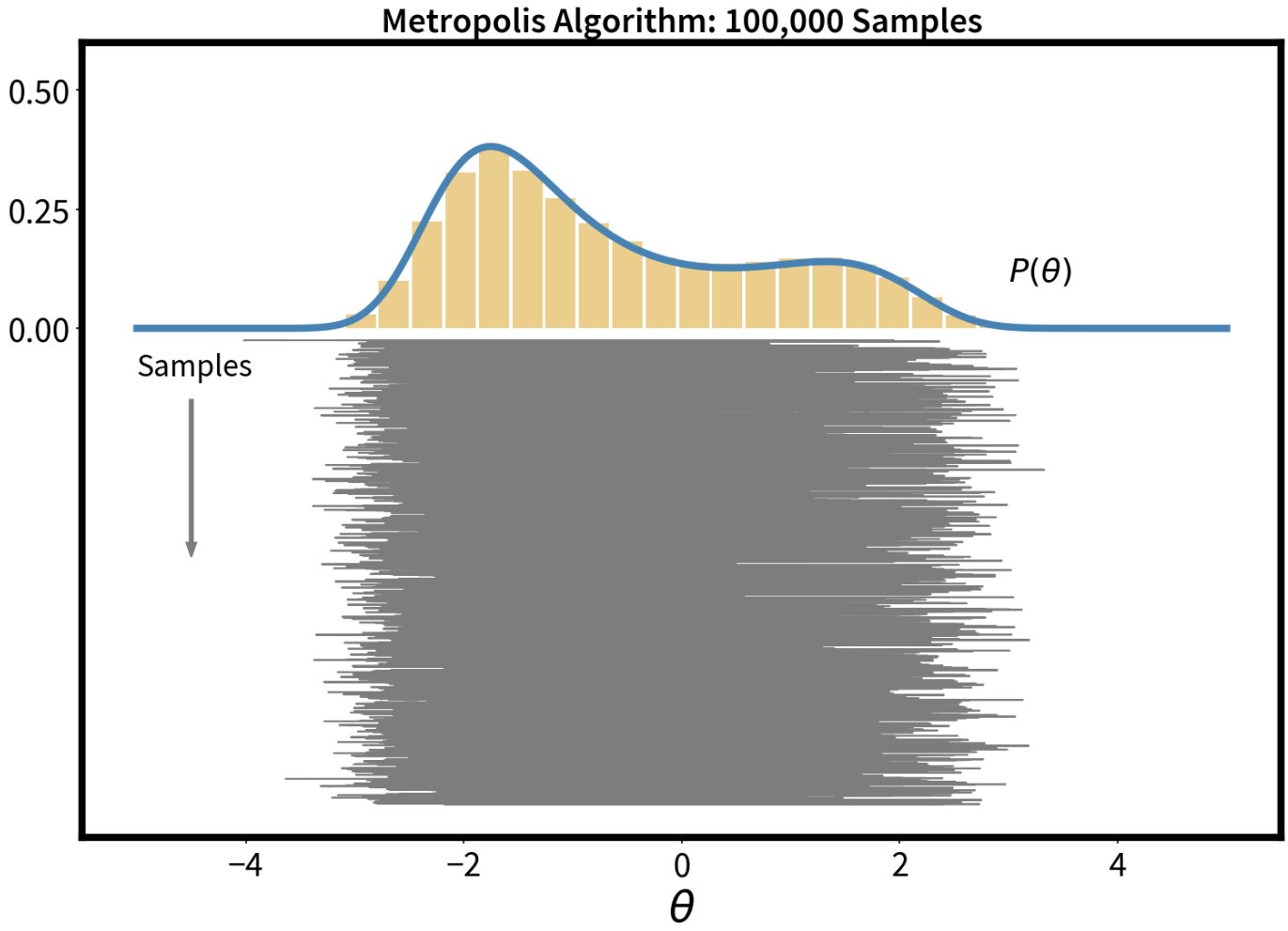
- The Metropolis algorithm assumes that the proposal distribution is symmetric: $Q\left(\theta^{'}|\theta^{(t)}\right) = Q\left(\theta^{(t)}|\theta^{'}\right)$
    - the asymmetric case is the Metropolis-Hastings algorithm (with the greyratio)
- We still need to specify $Q\left(\theta^{'}|\theta^{(t)}\right)$, and its length-scale r
- We must allow time for the chain to converge on its equilibrium distribution
    - the so-called burn in period
    - convergence is difficult to assess
- Successive samples are not independent!
    - it will be necessary to discard samples (*e.g.* retain every K-th)
    - how to do this is not straightforward to quantify

Metropolis Examples

Slide 12 The Metropolis Algorithm — Example

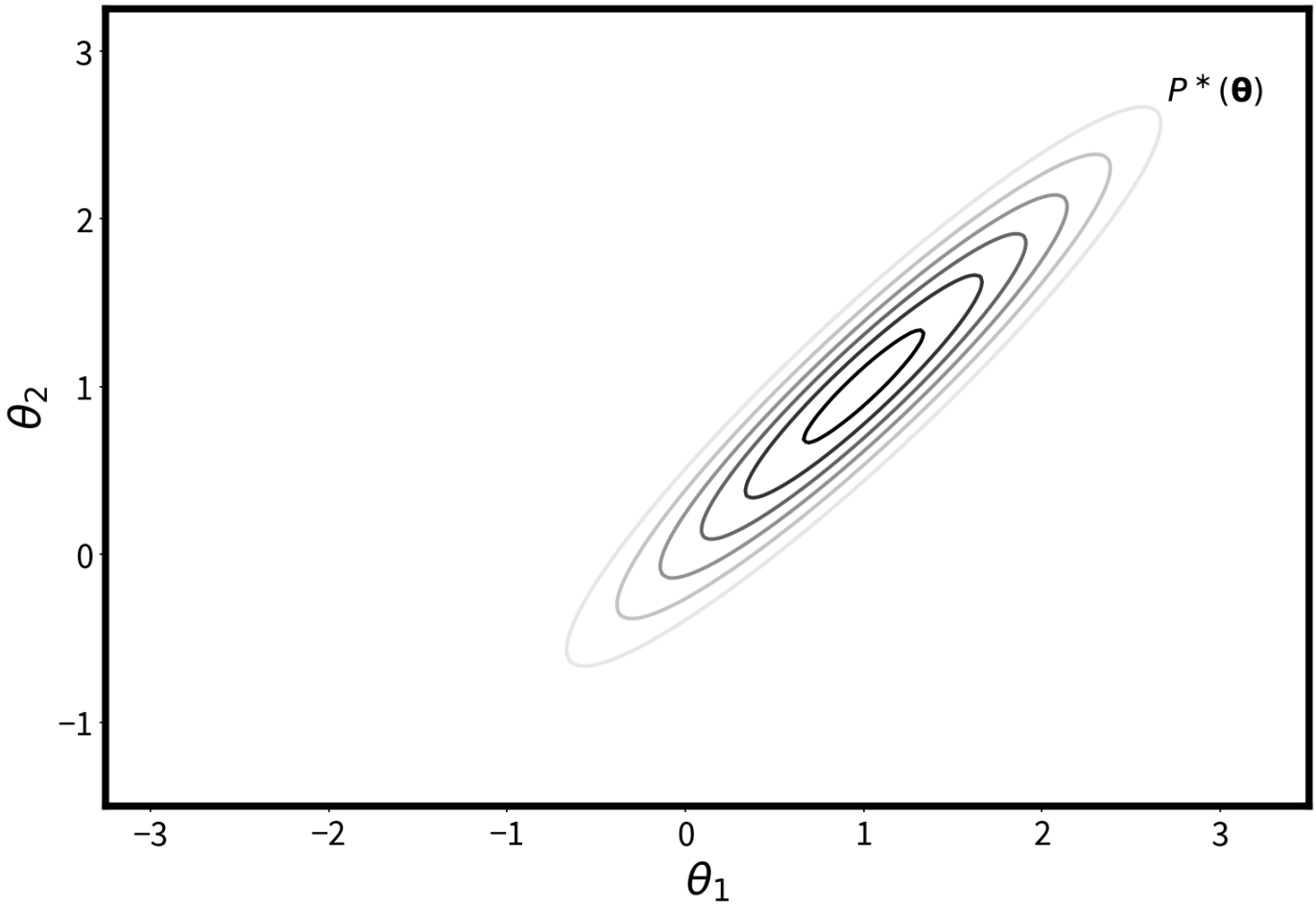



Metropolis Algorithm: 1,000 Samples
0.50
0.25
0.00
P(θ)
Samples
−4
−2
0
2
4
θ

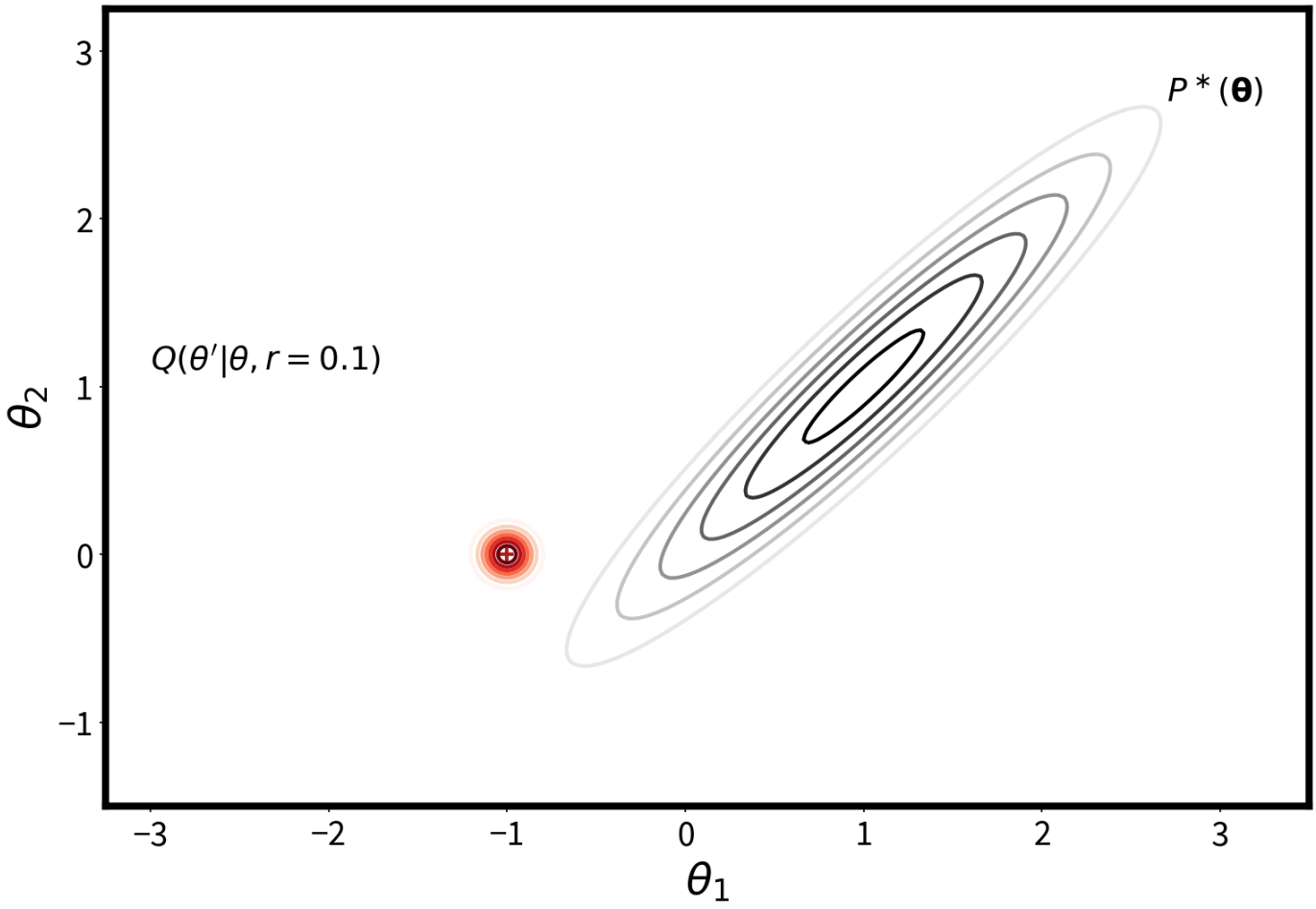


Slide 13 The Metropolis Algorithm — Higher Dimensions

Correlated example $P^*(\mathbf{\theta})$

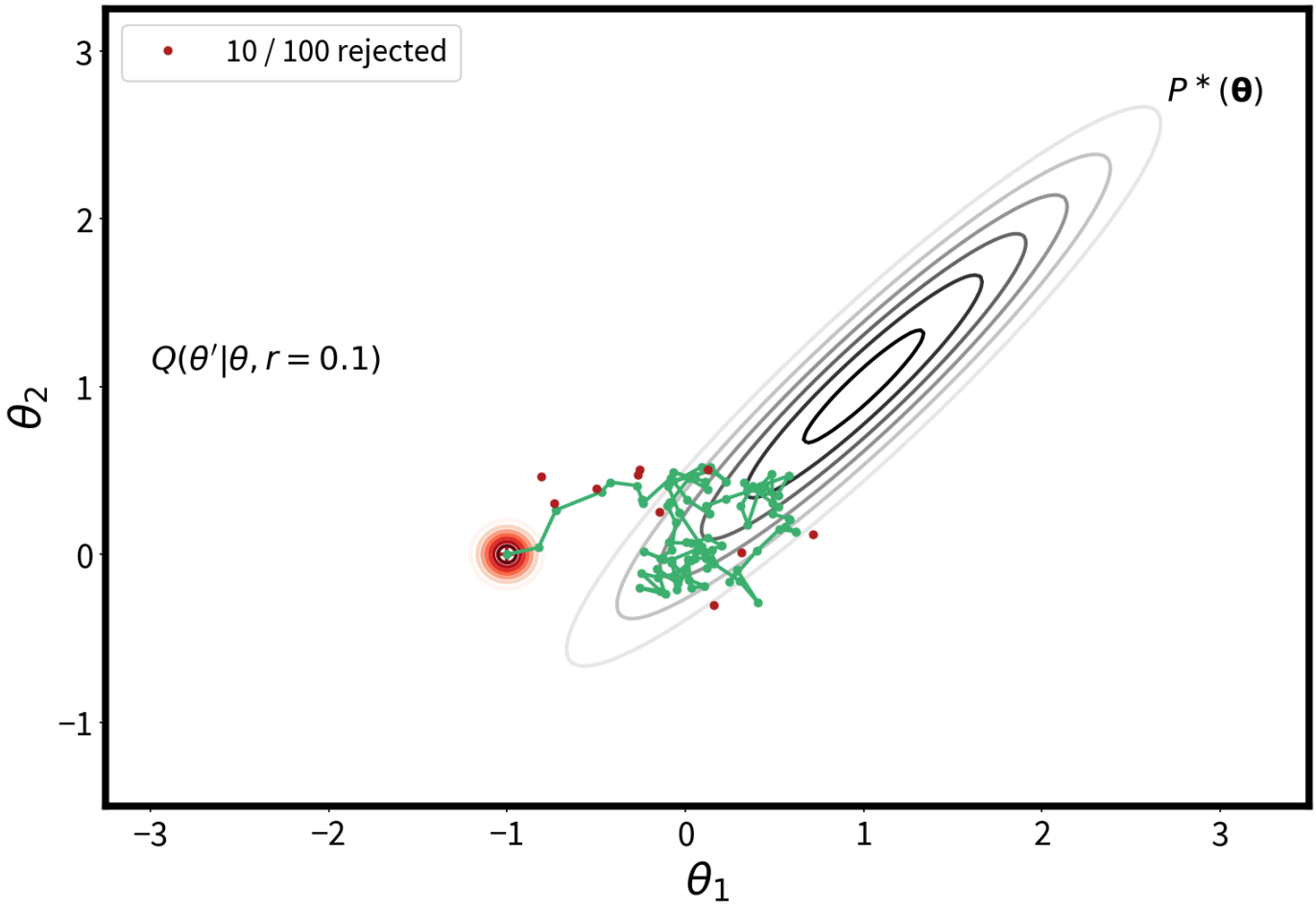


Correlated example $P^*(\mathbf{\theta})$ with proposal $Q\left(\theta'|\theta^{(t)}\right)$

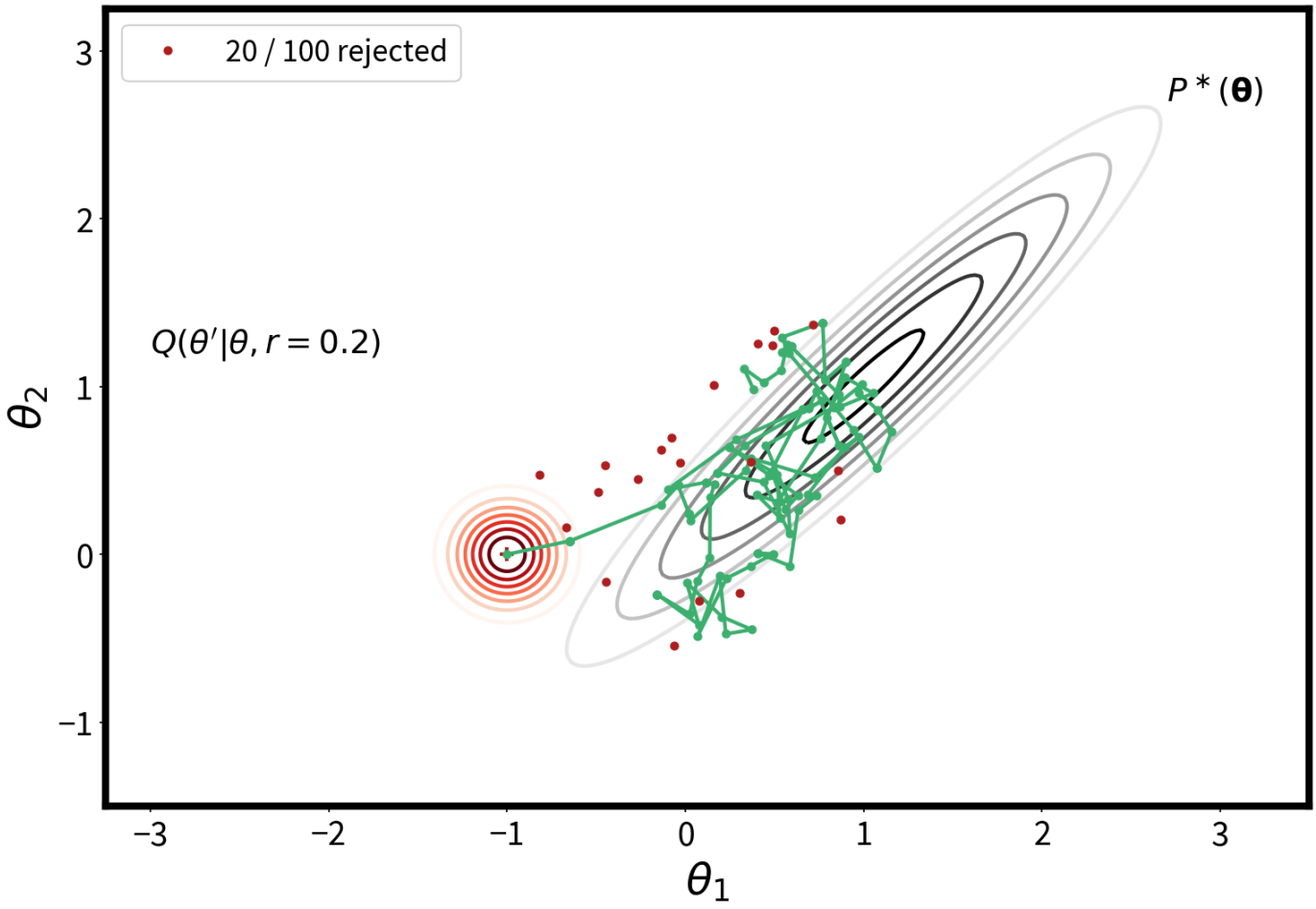


Slide 14 The Metropolis Algorithm — Higher Dimensions

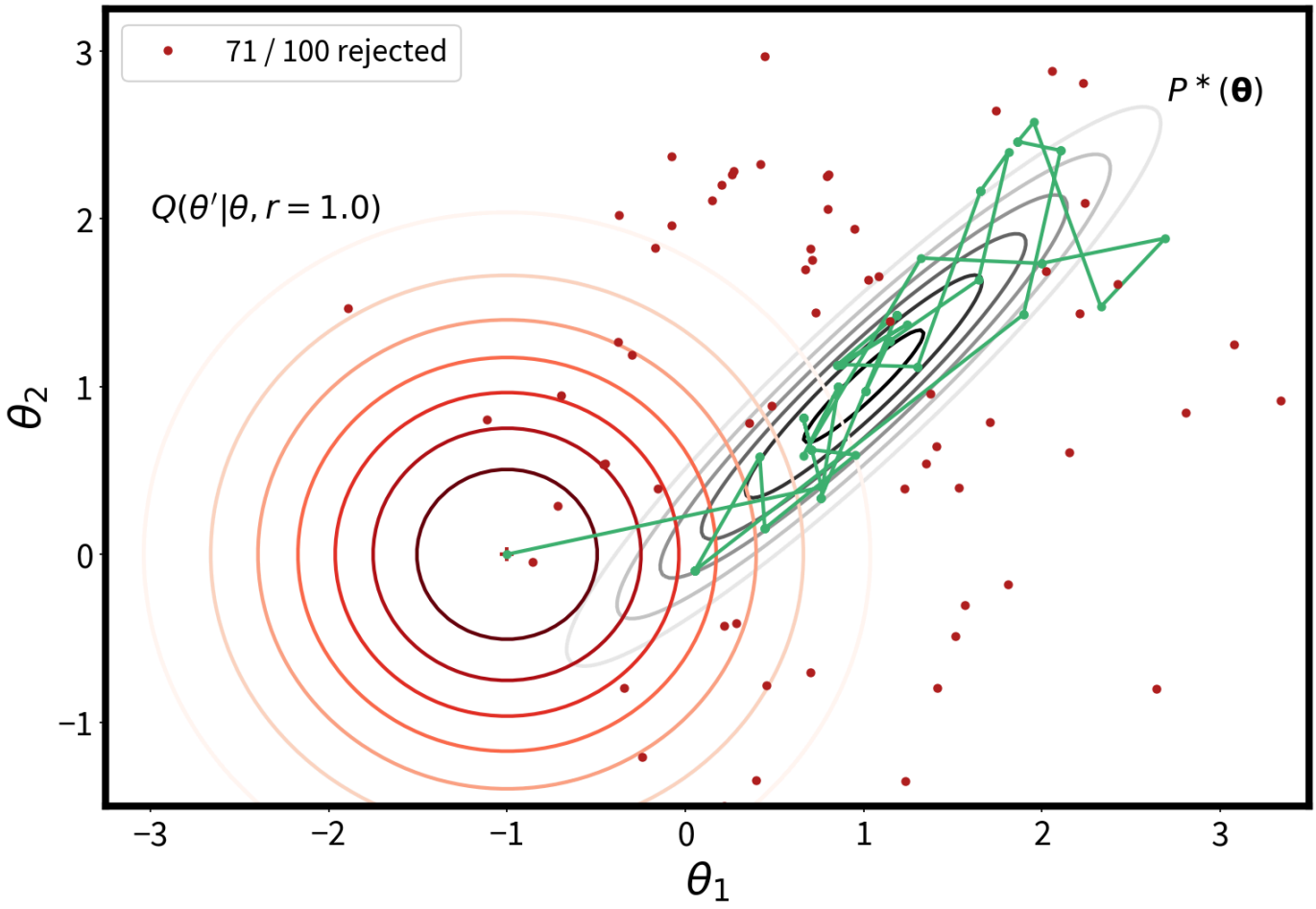
Length scale r universally too small: random walk behaviour



Length scale r too small in one direction: random walk in that direction



Length scale r too large in one direction: many rejected samples



Slide 15 **Metropolis-Hastings: Summary**

- Convergence can be problematic when length scales are significantly different along different directions in space
  - if r is too small, progress through state-space is a slow random walk
  - if r is too high, many samples will be rejected
- In high dimensions, if the rejection rate is not to become catastrophically high, r must be no greater than σ_{min}, the smallest length-scale of $P^*(\mathbf{\theta})$
- The number of steps L required to obtain a sample that may be considered independent of the start-point is:

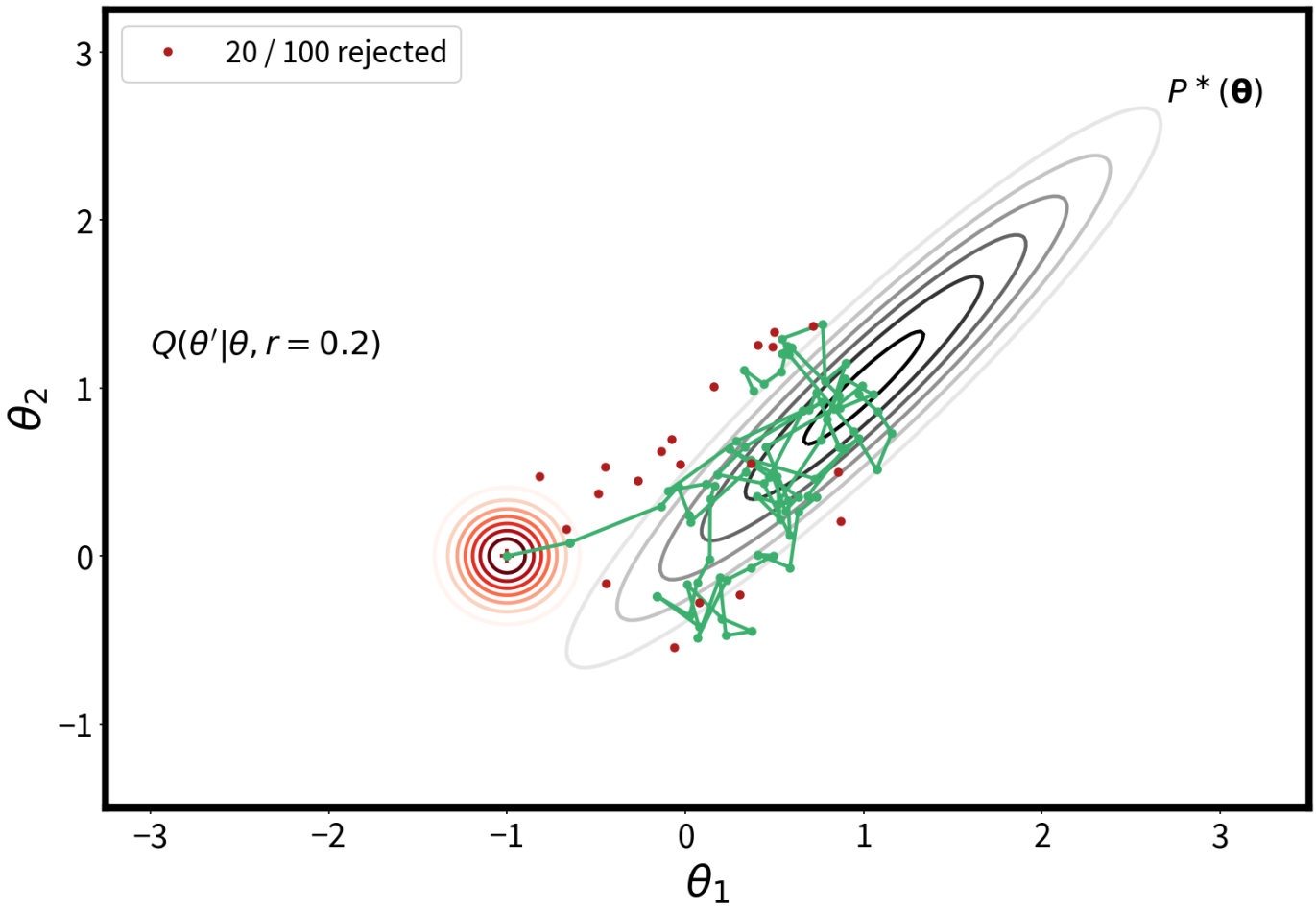
$$L \geq \left(\frac{\sigma_{\max}}{r} \right)^2$$

where $\sigma_{\max}$ is the largest length-scale of $P^*(\boldsymbol{\theta})$

- This may be a very long time!

Slide 16 Metropolis-Hastings: Length Scales

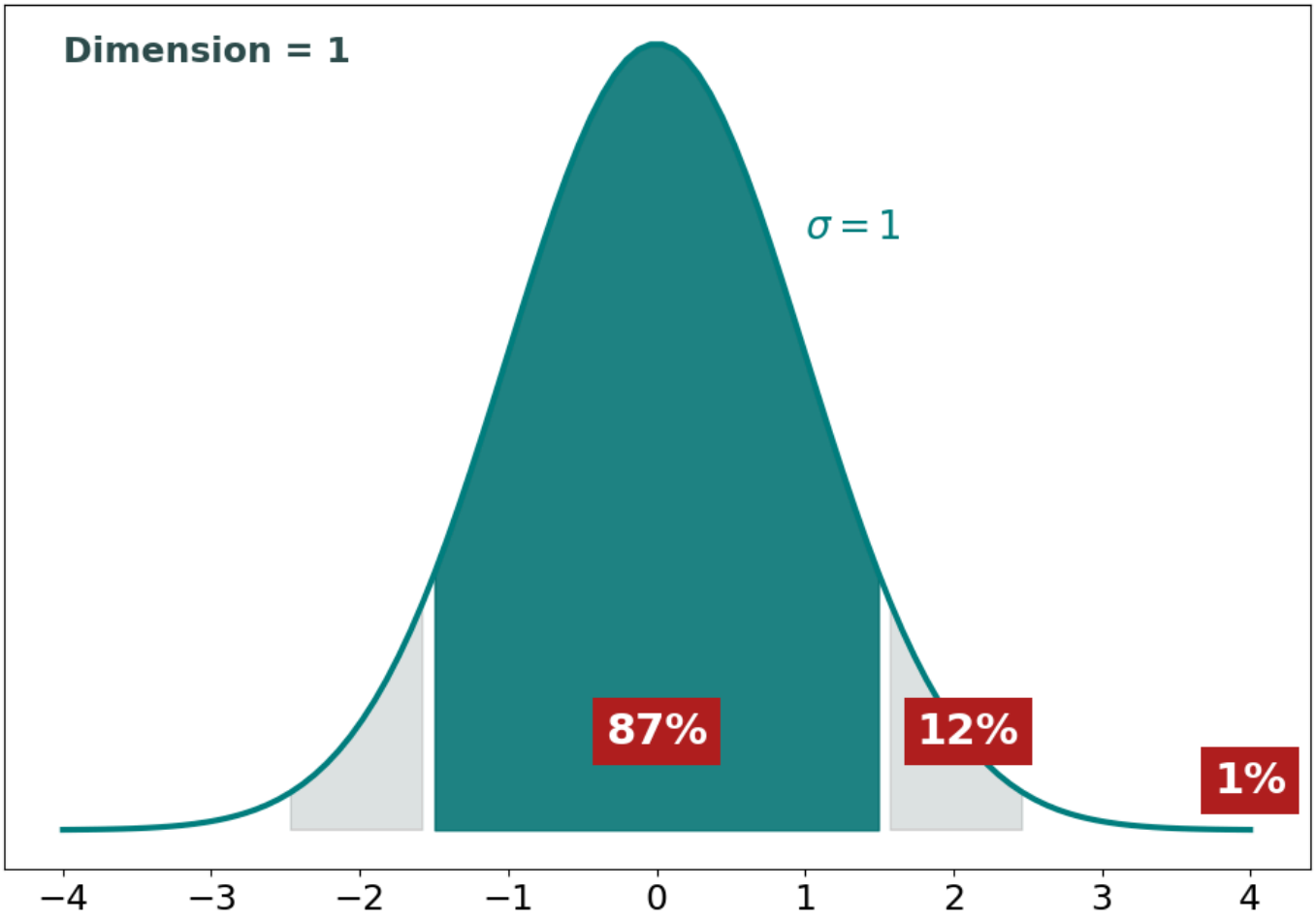
Length scale r matched to $\sigma_{\min}$:

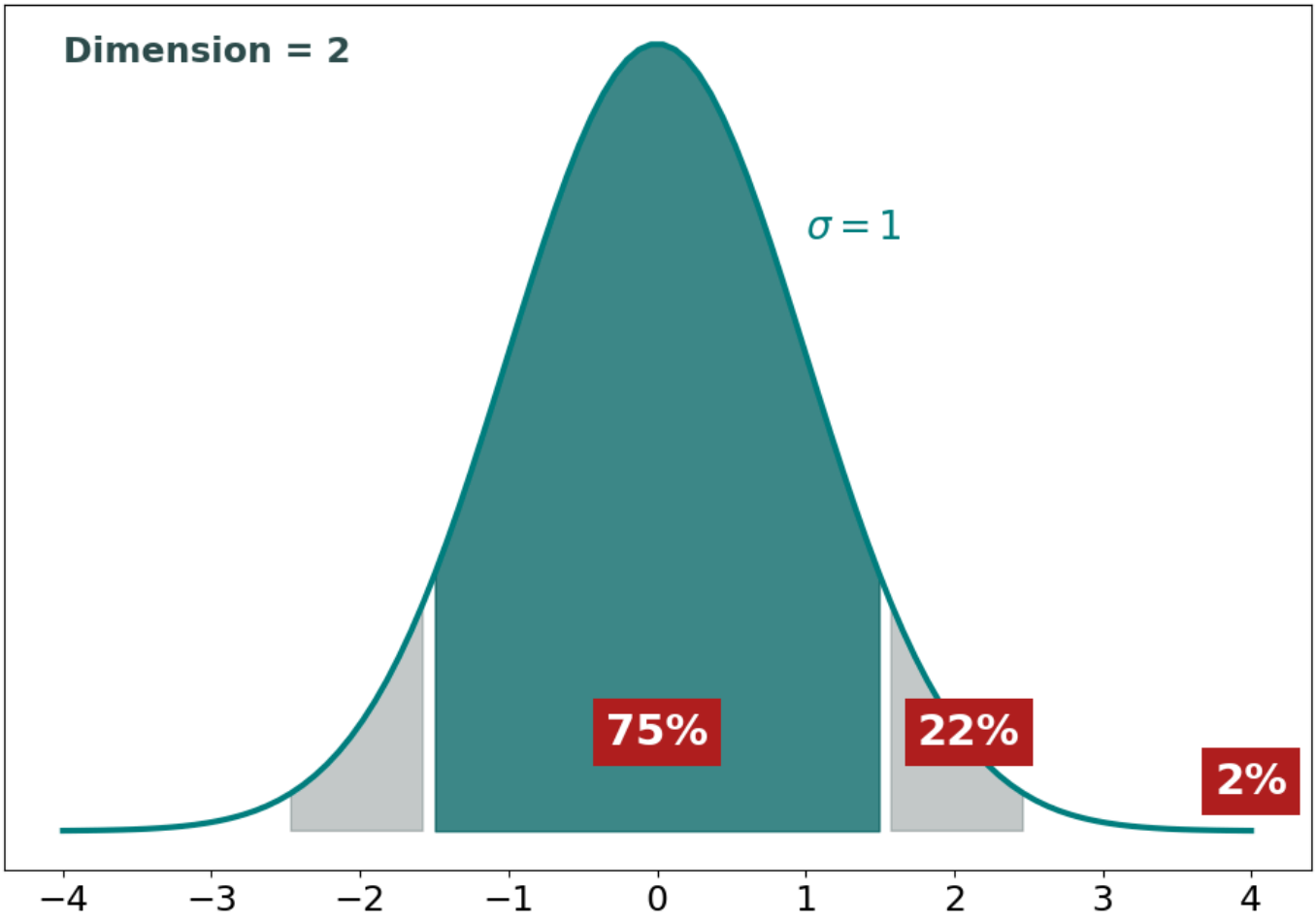


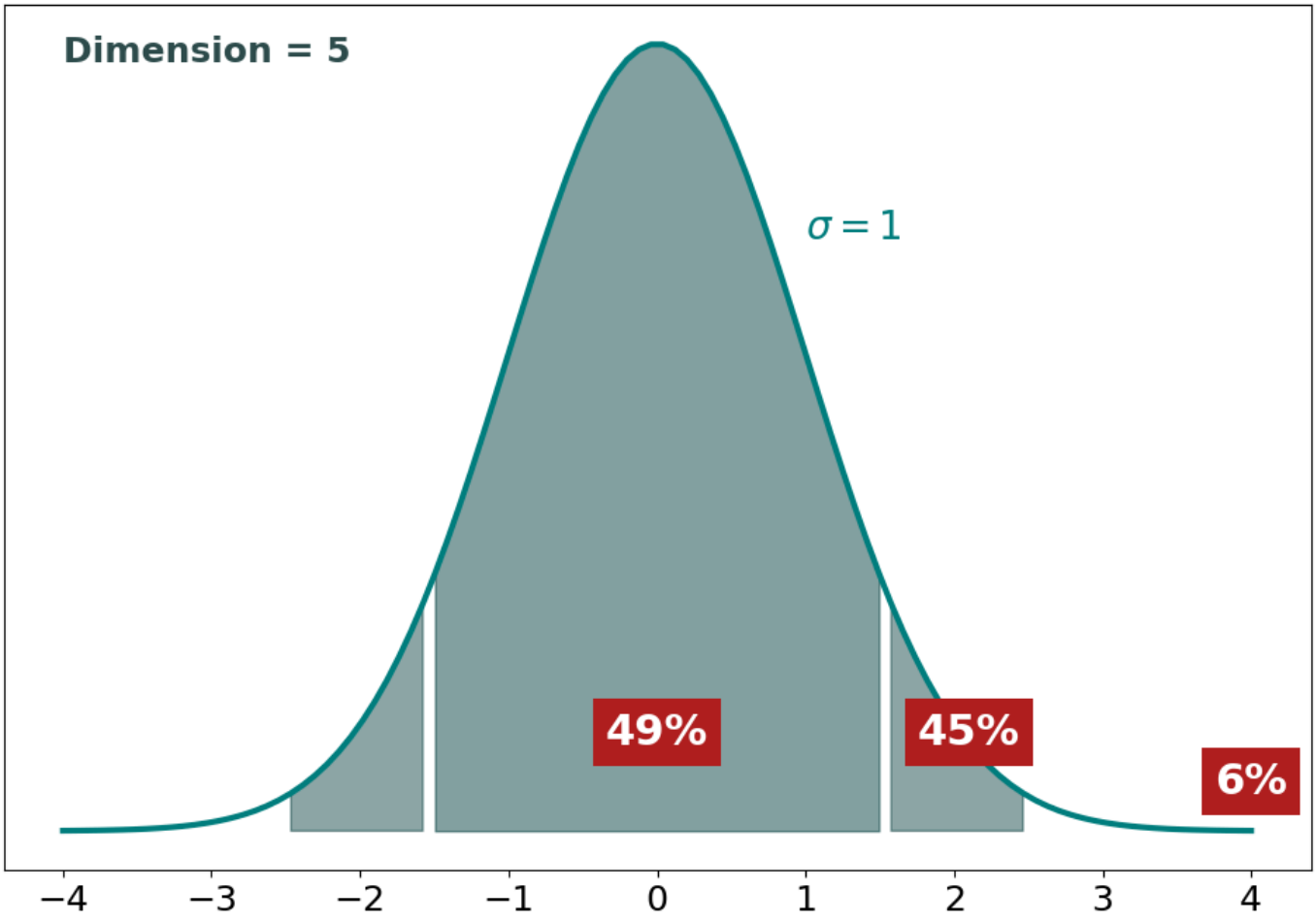
Hamiltonian Monte Carlo (HMC)

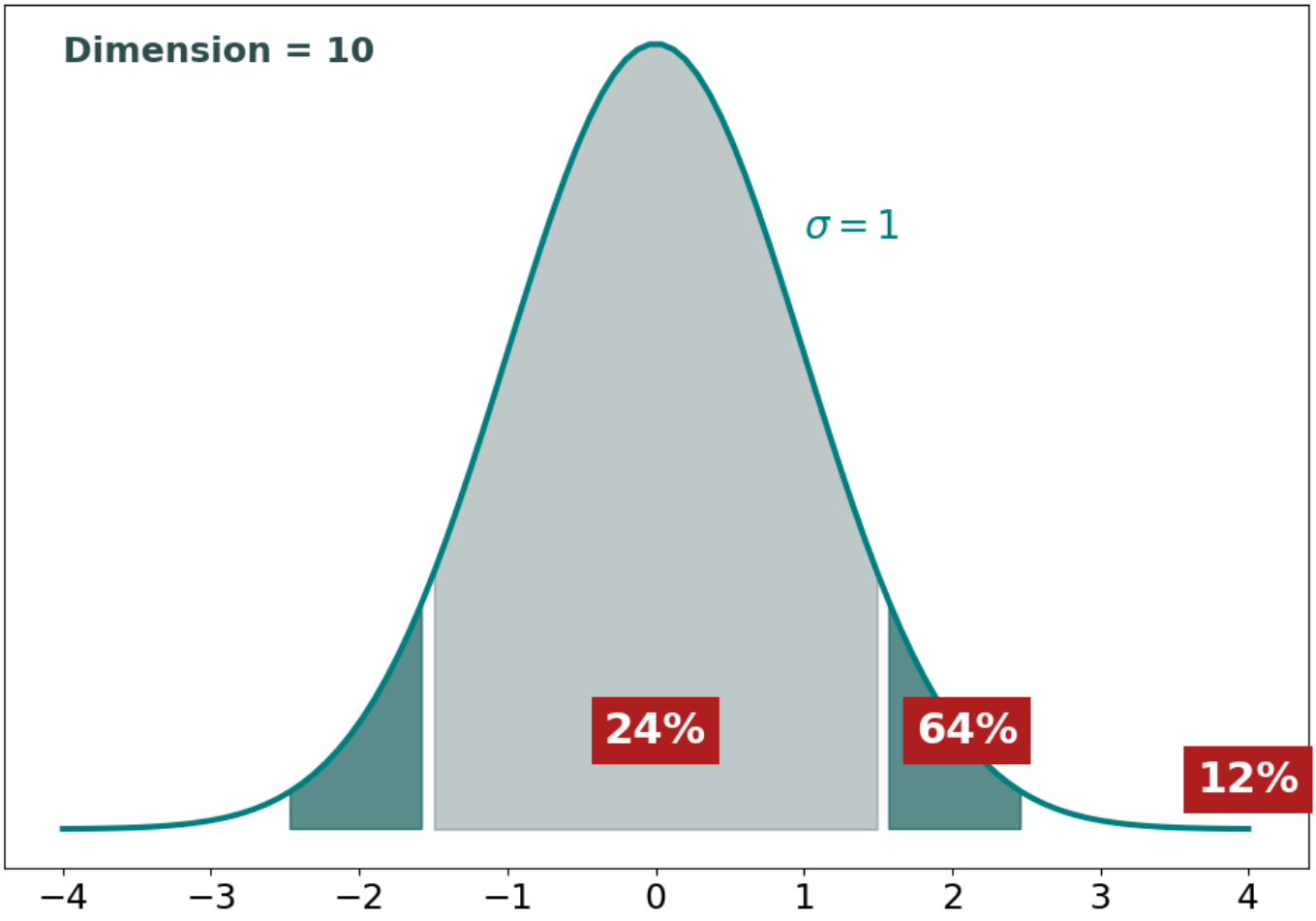
Slide 17 Distributions in High Dimensions

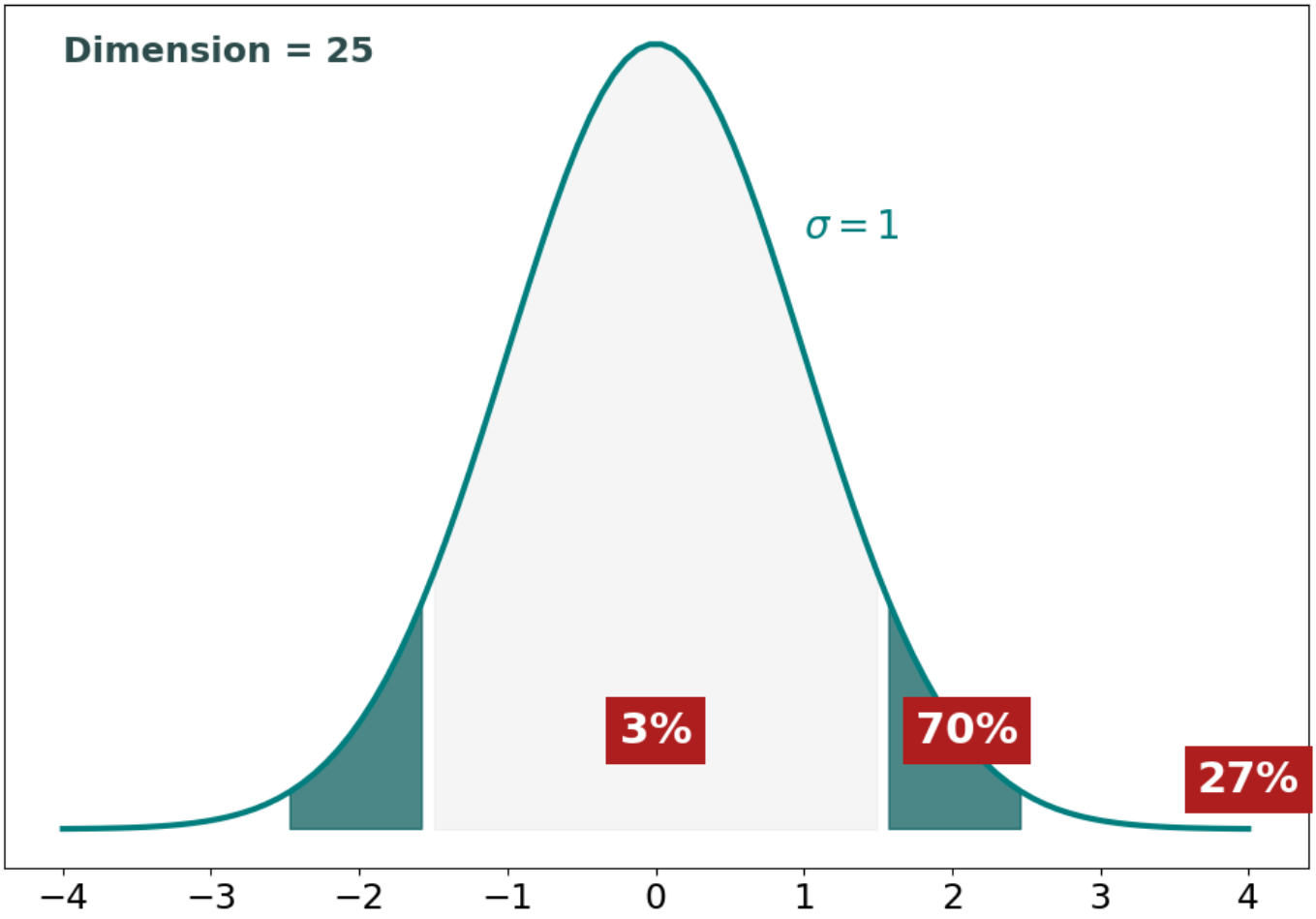
Where is the Gaussian probability mass?




Dimension = 2
$\sigma = 1$
75%
22%
2%
−4
−3
−2
−1
0
1
2
3
4


Dimension = 5
$\sigma = 1$
49%
45%
6%
−4
−3
−2
−1
0
1
2
3
4


Dimension = 10
$\sigma = 1$
24%
64%
12%
−4
−3
−2
−1
0
1
2
3
4



Slide 18 **Hamiltonian (or Hybrid) Monte Carlo**

May be considered “state of the art” in MCMC (and stochastic approximation)

- **Intuition:** Kicking a ball around the landscape $\log P^*(\boldsymbol{\theta})$, simulating its dynamics.
- Makes use of gradient information from $P^*(\boldsymbol{\theta})$: $\frac{\partial \log P^*(\boldsymbol{\theta})}{\partial \boldsymbol{\theta}}$
- Introduces momentum variables $\rho_i = \partial\theta_i/\partial t$
- Embeds $P^*(\boldsymbol{\theta})$ within a Hamiltonian function:

$$H(\boldsymbol{\theta}, \boldsymbol{\rho}) = -\log P^*(\boldsymbol{\theta}) + \frac{1}{2} \| \boldsymbol{\rho} \|^2$$

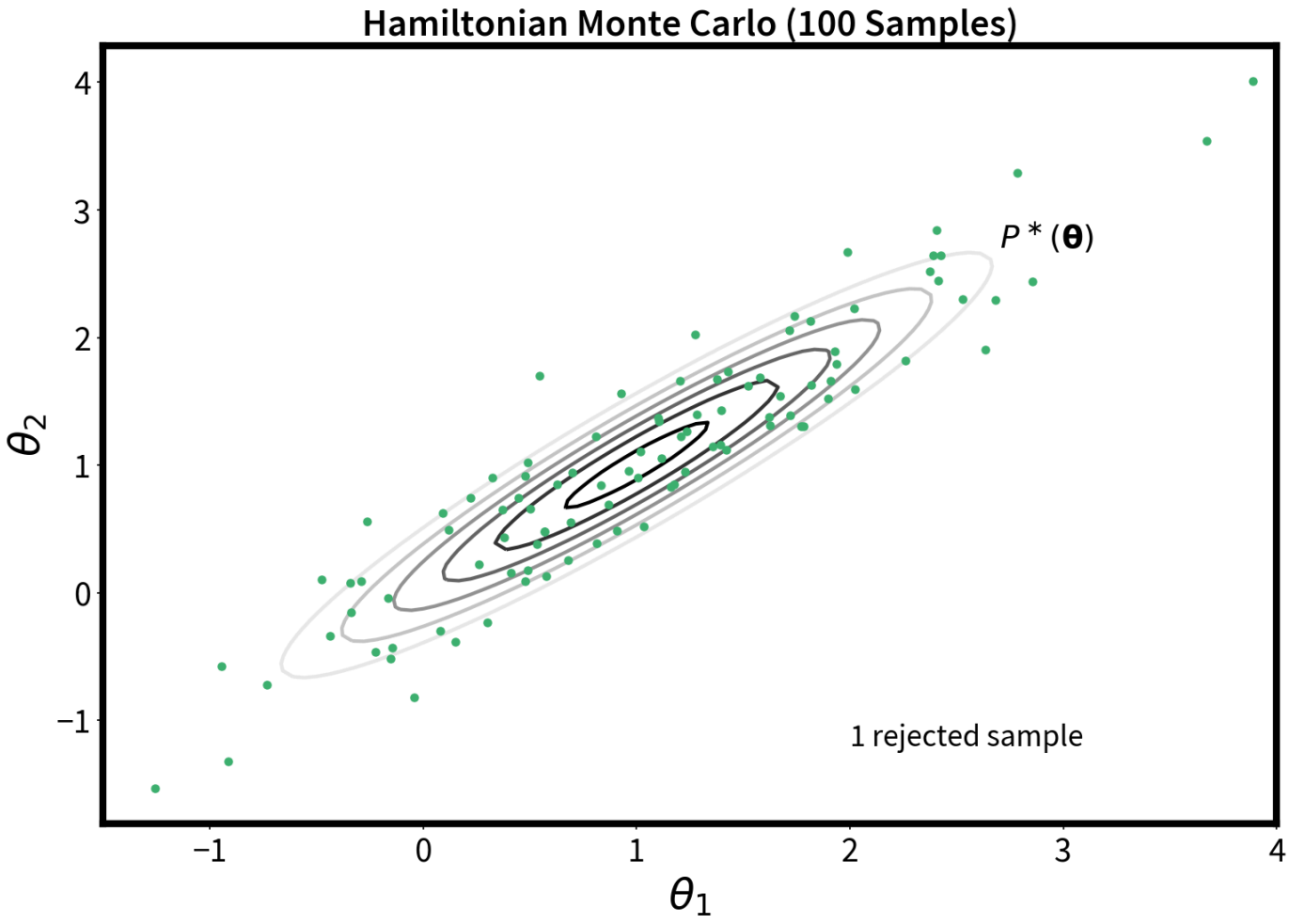
- A new state $\boldsymbol{\theta}'$ is proposed by randomly sampling $\boldsymbol{\rho}$ then iterating (numerically) the Hamiltonian dynamics ahead in time by several steps

- The Metropolis criterion is then applied to accept/reject the new state with probability

$$\min\left\{1, \exp\left[H\left(\boldsymbol{\theta}, \boldsymbol{\rho}\right) - H\left(\boldsymbol{\theta}', \boldsymbol{\rho}'\right)\right]\right\}$$

- The new state should be independent (no sub-sampling)

Slide 19 Hamiltonian Monte Carlo — Correlated Example



Slide 20 Hamiltonian Monte Carlo — Linear Regression

Return to the earlier Bayesian linear regression example:

- Use HMC to sample from the full posterior over α, σ and r
  - weights $\mathbf{w}$ are integrated out analytically as before
- This posterior is:

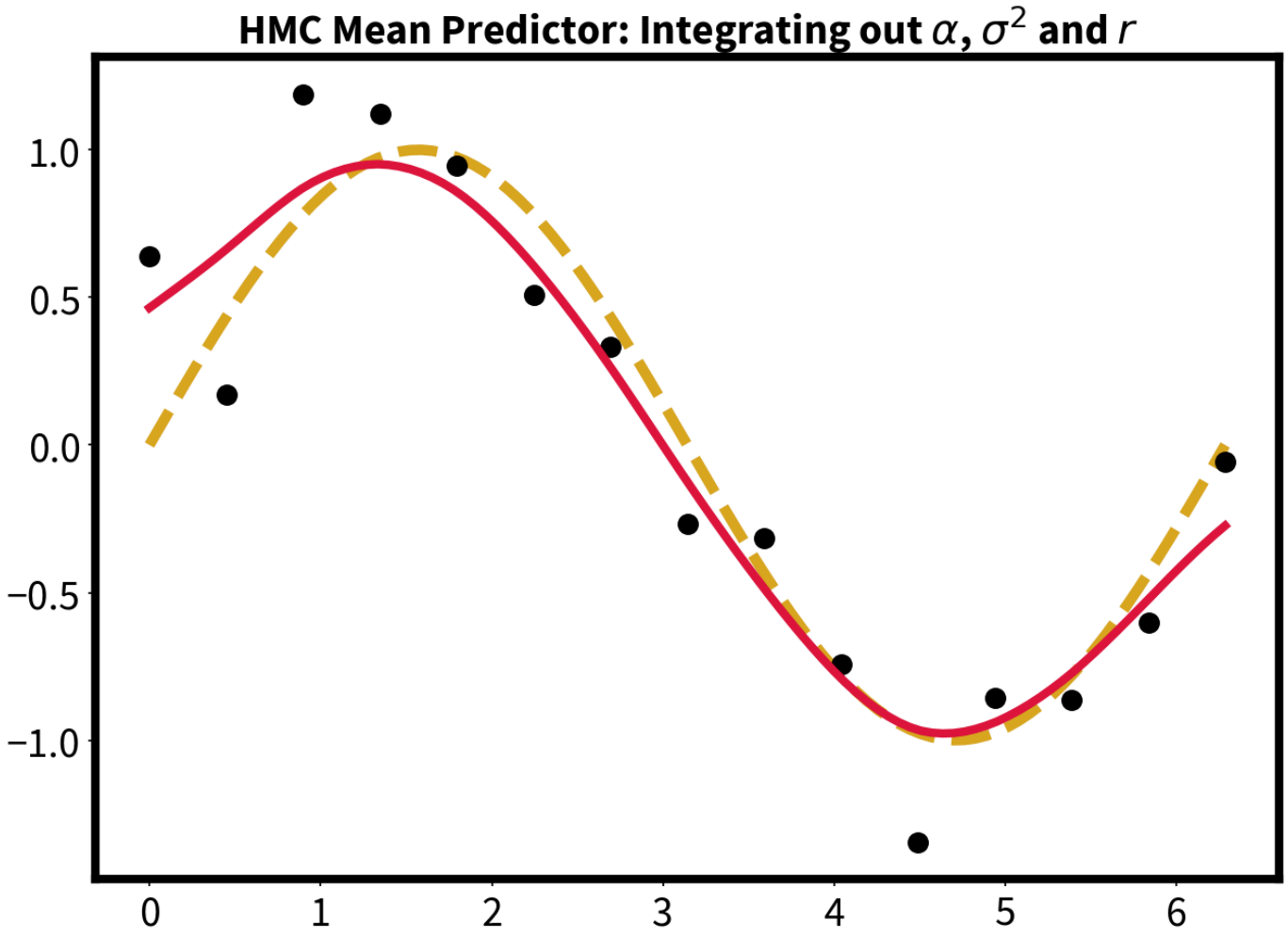
$$p\left(\alpha, \sigma^2, r|\mathbf{y}\right) = \frac{p\left(\mathbf{y}|\alpha, \sigma^2, r\right)\ p\left(\alpha\right)\ p\left(\sigma^2\right)\ p\left(r\right)}{p\left(\mathbf{y}\right)}$$

- The numerator defines our $P^*\left(\boldsymbol{\theta}\right)$:
  - $p\left(\mathbf{y}|\alpha, \sigma^2, r\right)$ is the marginal likelihood
  - assume log-uniform priors for $p\left(\alpha\right)$, $p\left(\sigma^2\right)$ and $p\left(r\right)$
- Use samples to approximate the predictive distribution:

$$p\left(y_*|\mathbf{y}\right) = \iiint p\left(y_*|\mathbf{y}, \alpha, \sigma^2, r\right)\ p\left(\alpha, \sigma^2, r|\mathbf{y}\right)\ d\alpha\ d\sigma^2\ dr$$

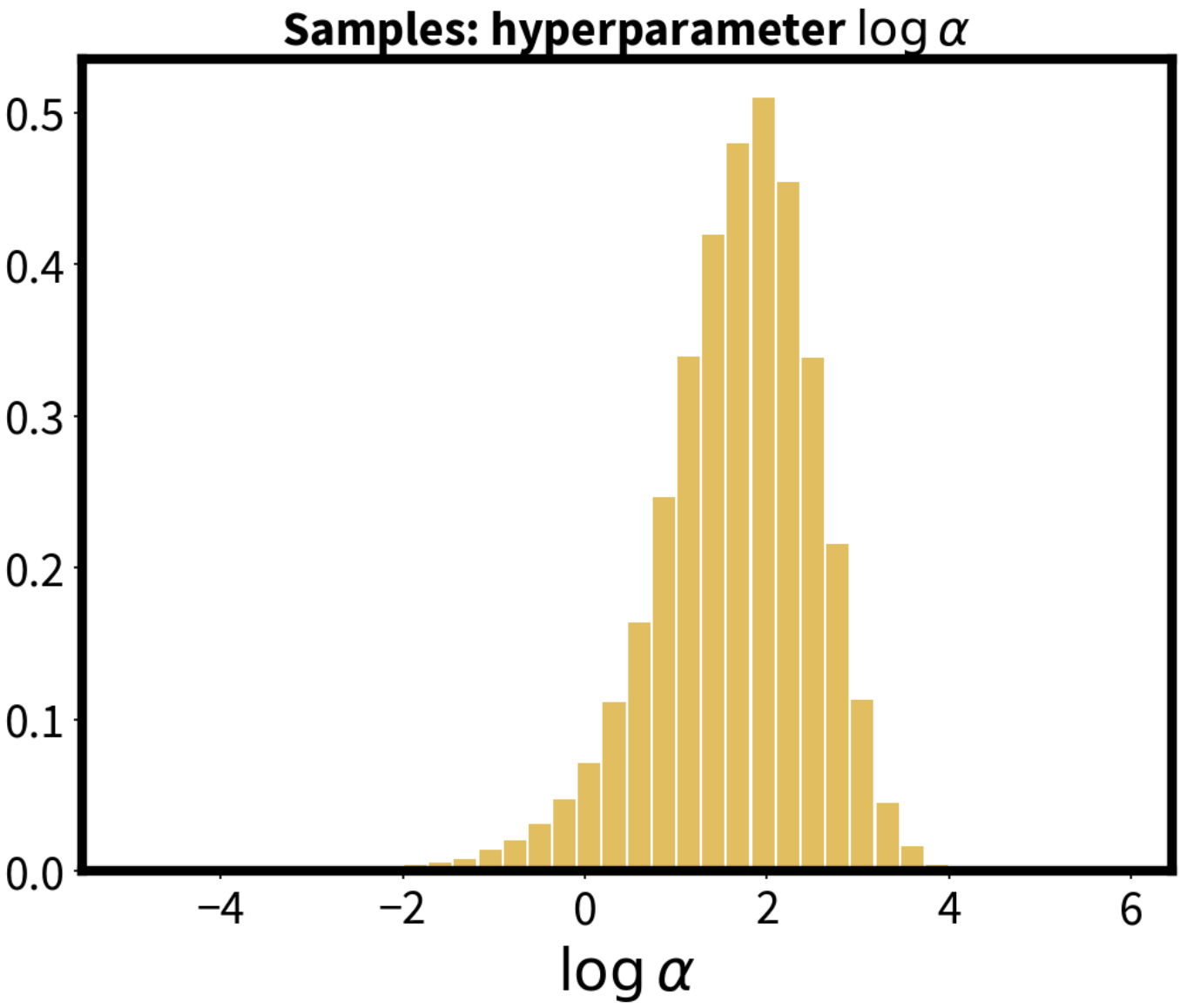
Slide 21 Hamiltonian Monte Carlo — Linear Regression The mean predic-

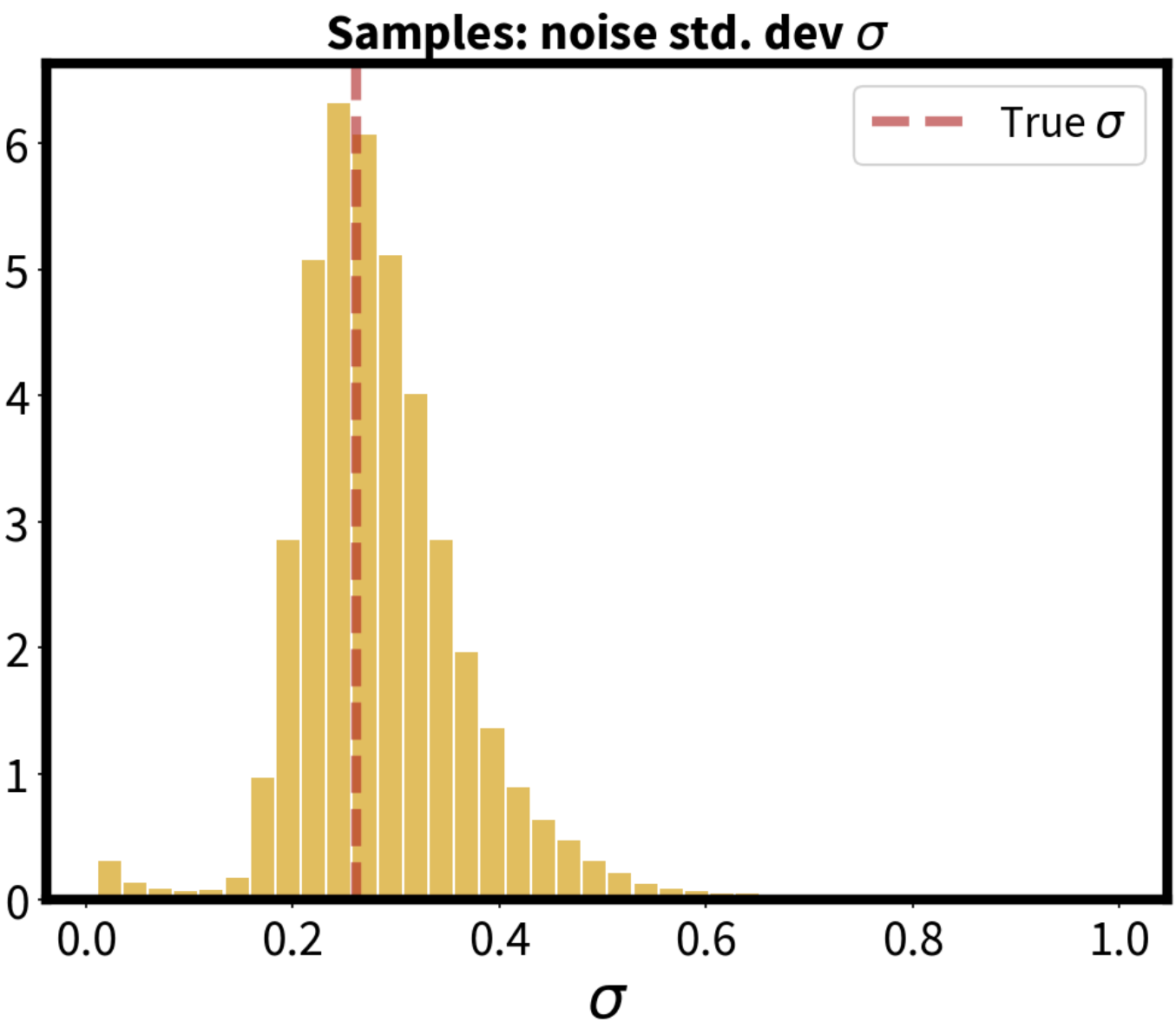
tor, averaged over $p\left(\alpha, \sigma^2, r|\mathbf{y}\right)$

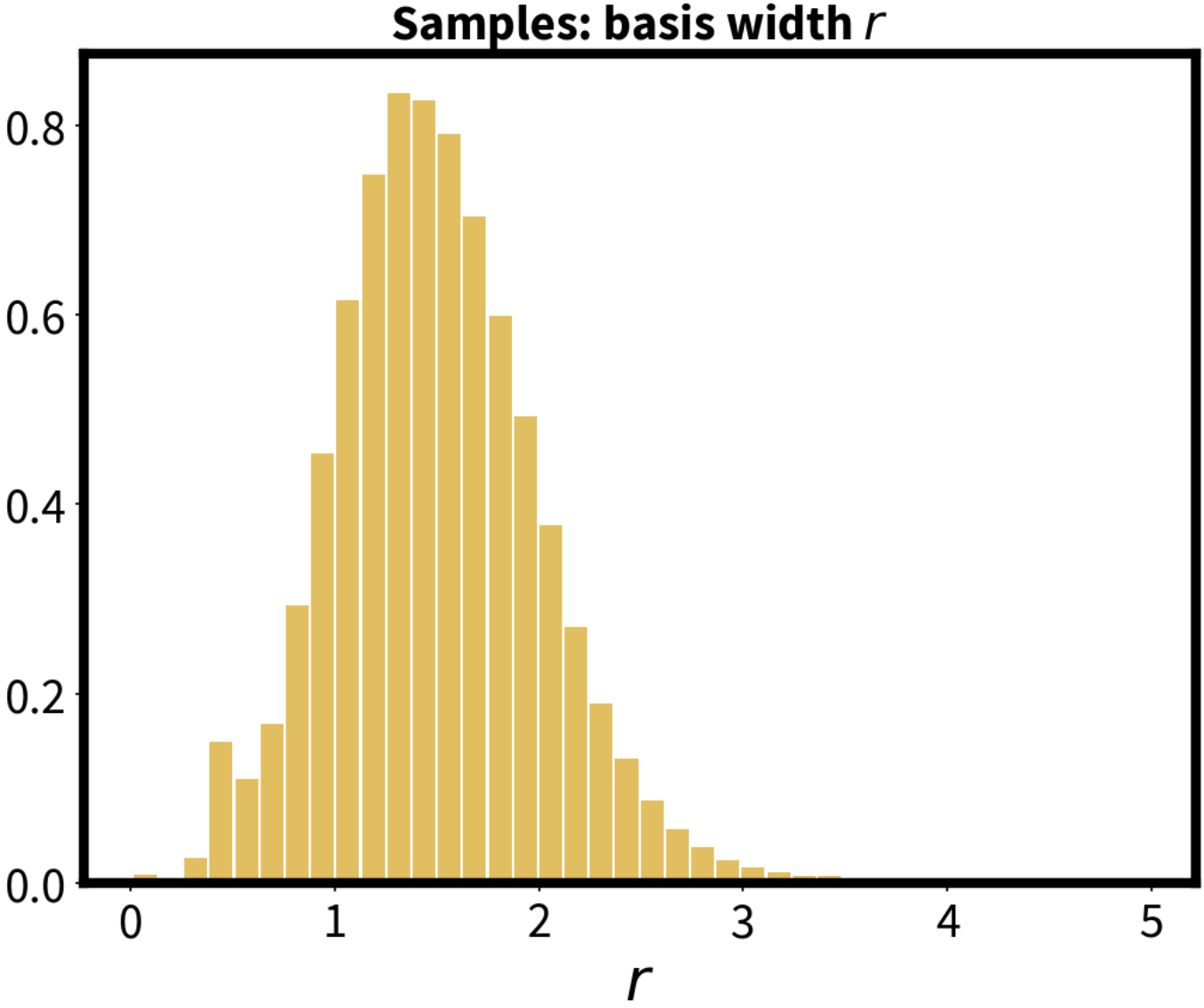


Slide 22 Hamiltonian Monte Carlo — Linear Regression Histograms of

marginal samples




Samples: noise std. dev σ
True σ
0
1
2
3
4
5
6
0.0
0.2
0.4
0.6
0.8
1.0
σ



Slide 23 **Hamiltonian Monte Carlo — Summary & Observations**

- HMC is effective at removing random walk behaviour, by exploiting the local shape of $\log P^{*}(\boldsymbol{\theta})$ via its gradient
- There is some “voodoo” regarding the implementation:
  - number of steps of dynamics simulation L
  - simulation step size (time-step) ϵ

- "simulation time" $\tau = L \times \epsilon$, the "distance travelled" across the $\log\ p(\boldsymbol{\theta})$ landscape
- large ϵ means less accurate dynamics (more rejected samples)
- desired acceptance rate (65%)
- random perturbations to ϵ
- re-sampling of momentum $\boldsymbol{\rho}$, and use of momentum "persistence"
- random inversion of direction of "time"

- These factors may require some tuning to the task at hand
- Alternatively … use NUTS, the No U-turn Sampler

Slide 24 HMC Demo — Next Lecture

At the start of the next lecture (it's quite long), we will see HMC implemented from scratch in Python.

Papers & Software

Slide 25 Papers on How HMC Works

There are two excellent papers, introductory, yet not lacking detail:

- "MCMC using Hamiltonian Dynamics", Radford M. Neal [https://arxiv.org/abs/1206.1901]

- “A Conceptual Introduction to Hamiltonian Monte Carlo”, M. Betancourt [https://arxiv.org/abs/1701.02434]

Slide 26 **Software for Bayesian Inference & MCMC?**

There is no MCMC software in the ‘standard’ *Python* data science distributions, but some other libraries to seriously consider are:

- `Stan` (interfaces with *Python* via PyStan) https://pystan.readthedocs.io



- `PyMC` (native *Python*) `http://pymc.io/`



- `numpyro` (native *Python*) `https://num.pyro.ai`

